

Kernel PWN Walkthrough

Binary: Kernel 5.15.137 (vroot/hroot) | Analysis: statisch | Schwierigkeit: Elite

Was ist Static Analysis beim Kernel-PWN?

In Labs 01–04 haben wir Bugs **ausgenutzt**. Hier drehen wir den Blick um: Wir finden dieselben Bug-Klassen **statisch** – ohne den Kernel zu booten und ohne Oops. Real werden Kernel-Bugs in erster Linie von statischen Analysen oder von Fuzzern gefunden; der Exploit folgt danach. Beim Kernel-PWN ist dieser Schritt also der **Beginn**, nicht das Nachspiel.

Die Schwachstellen-Klassen, die wir jagen

```
vroot: char buf[64]; raw_copy_from_user(buf, arg, 512);    // 512 > 64 -> Stack-Overflow
hroot: kfree(g_v); ... g_v->handler(arg);                // UAF call-through
```

Die vier Werkzeuge im Überblick:

Werkzeug	Schärfe	vroot-Overflow?	hroot-UAF?
naive Regex	syntaktisch	ja (Heuristik)	nein
sparse (C=1)	Typen/Annotations	nein	nein
smatch	Datenfluss	ja	teils (schwer)
Coccinelle	Semantik	ja	ja (mit Trick)

Skript 1 – naive Regex-Heuristik (find-overflows.py)

Vollständig verwendeter Code:

```
#!/usr/bin/env python3
"""find-overflows.py - lightweight static scanner for stack overflows.

Looks for the exact bug class of vroot.c:

    char buf[64];
    ...
    copy_from_user(buf, (void __user *)arg, 512);    // 512 > 64 -> overflow

Heuristic, dependency-free and deliberately naive: a "real" tool would use
sparse/smatch/coccinelle (see the .cocci file + run-sparse.sh). The point is
to see why simple text matching finds vroot but a heap UAF (hroot) needs a
completely different analysis.

Usage:
    ./find-overflows.py FILE... (default: ../../shared/vroot/vroot.c)
"""
import re
import sys

CALL_KW = re.compile(r"copy_(?:from|to)_user\s*\(")
DECL_RE = re.compile(r"\b(?:char|unsigned\s+char|signed\s+char|int|u8|u16|"
                    r"u32|u64|long|short)\s+(\w+)\s*\[\s*(\d+)\s*\]\s*;"")

def top_level_parts(args):
    """split a call argument string on commas that are not nested in ( )"""
    parts, depth, cur = [], 0, ""
    for ch in args:
        if ch == "(":
            depth += 1
        elif ch == ")":
            depth -= 1
        if ch == "," and depth == 0:
            parts.append(cur)
            cur = ""
        else:
            cur += ch
    parts.append(cur)
    return [p.strip() for p in parts]
```

```

def find_calls(text):
    """yield (call_start_line, args_string) for every copy_*_user call"""
    for m in CALL_KW.finditer(text):
        i = m.end()
        depth = 1
        while i < len(text) and depth:
            if text[i] == "(":
                depth += 1
            elif text[i] == ")":
                depth -= 1
            i += 1
        if depth == 0:
            line = text[:m.start()].count("\n") + 1
            yield line, text[m.end():i - 1]

def scan(path):
    with open(path, "r", errors="replace") as f:
        text = f.read()
        decls = {m.group(1): int(m.group(2))
                  for m in DECL_RE.finditer(text)}

    hits = 0
    for line, args in find_calls(text):
        parts = top_level_parts(args)
        if len(parts) != 3:
            continue
        dst, _, size = parts
        if dst not in decls:
            continue
        if not size.isdigit():
            continue # dynamic size: can't decide without dataflow
        n = int(size)
        if n > decls[dst]:
            hits += 1
            print(f"{path}:{line}: STACK OVERFLOW candidate: "
                  f"{dst}[{decls[dst]}] but copy_*_user copies {n} bytes")
    if not hits:
        print(f"{path}: no obvious stack-overflow matches")
    return hits

def main():
    files = sys.argv[1:] or ["../../shared/vroot/vroot.c"]
    bad = sum(scan(f) for f in files)
    sys.exit(1 if bad else 0)

if __name__ == "__main__":
    main()

```

Ausführung:

```

cd ../05-elite/static-analysis
./find-overflows.py ../../shared/vroot/vroot.c
# -> STACK OVERFLOW candidate: buf[64] but copy_*_user copies 512 bytes
./find-overflows.py ../../shared/hroot/hroot.c
# -> no obvious stack-overflow matches (UAF wird NICHT gefunden)

```

Warum findet es vroot, aber nicht hroot? Es matcht `copy_*_user` als **Substring** (`raw_copy_from_user` enthält `copy_from_user`) und vergleicht eine konstante Größe gegen die Deklaration. Für den UAF gibt es kein solches Muster – der braucht Pointer-Lifecycle-Analyse.

Skript 2 – Coccinelle vroot-Overflow (vroot-overflow.cocci)

Die überarbeitete Regel (nutzt den Cross-Case-Trick, da vroot den Aufruf im `switch/case` hat):

```

// vroot-overflow.cocci
//
// Coccinelle rule: flag copy_from_user() into a fixed-size stack array where
// the copy size is a constant LARGER than the array.
//

```

```

// This catches the vroot.c bug:
//   char buf[64];
//   ...
//   raw_copy_from_user(buf, (void __user *)arg, 512);   // 512 > 64
//
// Usage:
//   spatch --sp-file vroot-overflow.cocci --dir ../../shared
//   or single file:
//   spatch --sp-file vroot-overflow.cocci ../../shared/vroot/vroot.c
//
// DESIGN NOTE: vroot.c declares buf at the top of the function but calls
// raw_copy_from_user() inside a switch/case. A single sequential rule can't
// traverse the switch, so we use two INDEPENDENT rules (a declaration and a
// call-site match) and correlate them in Python by buffer name -- the same
// cross-case trick as hroot-uaf.cocci.
//
// API NOTE: vroot.c calls raw_copy_from_user() (low-level primitive), not
// copy_from_user(). A semantic matcher must name the exact function. This is
// why naive text-matching (find-overflows.py finds it as the substring
// "copy_from_user" inside "raw_copy_from_user") is a different, less precise
// animal.

@decl@
identifier buf;
constant N;
@@
char buf[N];

@copycall@
identifier buf2;
expression src;
constant SZ;
@@
copy_from_user(buf2, src, SZ)

@copycall_raw@
identifier buf2;
expression src;
constant SZ;
@@
raw_copy_from_user(buf2, src, SZ)

@script:python depends on decl && copycall@
buf << decl.buf;
N << decl.N;
buf2 << copycall.buf2;
SZ << copycall.SZ;
@@
if str(buf) == str(buf2) and N.isdigit() and SZ.isdigit() and int(N) < int(SZ):
    print("stack buffer %s[%s] but copy_from_user copies %s bytes -> overflow"
          % (buf, N, SZ))

@script:python depends on decl && copycall_raw@
buf << decl.buf;
N << decl.N;
buf2 << copycall_raw.buf2;
SZ << copycall_raw.SZ;
@@
if str(buf) == str(buf2) and N.isdigit() and SZ.isdigit() and int(N) < int(SZ):
    print("stack buffer %s[%s] but raw_copy_from_user copies %s bytes -> overflow"
          % (buf, N, SZ))

```

Ausführung:

```

spatch --sp-file vroot-overflow.cocci ../../shared/vroot/vroot.c
# -> stack buffer buf[64] but raw_copy_from_user copies 512 bytes -> overflow

```

Skript 3 – Coccinelle hroot-UAF (hroot-uaf.cocci)

Die eigentliche Aufgabe: Eine Regel, die den UAF ansatzweise findet. Da kfree und der ->handler-Aufruf in verschiedenen switch/case-Blöcken liegen, kann ein sequentielles Pattern den switch nicht durchlaufen. Lösung: zwei unabhängige

Regeln + Korrelation per Variablennamen im Python-Teil. Vollständiger Code:

```
// hroot-uaf.cocci -- heuristic for the hroot use-after-free (Lab 04 / 05)
//
// hroot.c frees its heap object via kfree(g_v) but NEVER assigns
// g_v = NULL afterwards. A later g_v->handler(arg) is the UAF call-through.
//
// Because kfree() and the ->handler() call live in *different* switch/case
// blocks, a single sequential rule CANNOT traverse the switch (Coccinelle
// stops at case labels). So we use two INDEPENDENT rules that match anywhere
// in the file and correlate them in Python by symbol name. This is the
// "cross-case / cross-function" approach the README talks about.
//
// Usage:
//   spatch --sp-file hroot-uaf.cocci ../../shared/hroot/hroot.c

@kfree_site@
expression E;
@@
kfree(E)

@use_site@
expression E2;
@@
E2->handler(...)

@script:python@
E << kfree_site.E;
E2 << use_site.E2;
@@
if str(E) == str(E2):
    print("POSSIBLE UAF: '%s' is kfree()d and then ->handler(...) is "
          "called on the same pointer -- verify it is NOT NULLed in between"
          % E)
```

Ausführung:

```
spatch --sp-file hroot-uaf.cocci ../../shared/hroot/hroot.c
# -> POSSIBLE UAF: 'g_v' is kfree()d and then ->handler(...) is called
#   on the same pointer -- verify it is NOT NULLed in between
```

Das ist der „Cross-Function/Cross-Case“-Ansatz: Statt Sequenz-Matching erkennen wir beide Stellen global und prüfen in Python, ob es dieselbe Variable (g_v) ist.

sparse (make C=1)

sparse prüft nur Typen/Annotations, nicht die Größe eines Buffers gegen die Copy-Länge. Deshalb findet es den 64-vs-512-Overflow **nicht**. Das ist genau der pädagogische Punkt: ein Typ-Checker ist kein Bug-Finder.

```
bash run-sparse.sh ../../01-beginner/linux-5.15.137
# -> sparse found nothing about the 64-byte stack overflow
```

Warum Text-Matching für Datenfluss nicht reicht

- Text kennt keine Semantik: raw_copy_from_user ist eine andere Funktion als copy_from_user – der Regex sieht das nicht.
- Kein Kontext: falsche Treffer und übersehene echte Bugs.
- Keine Pointer-Lifecycle-Analyse: „kfree(p); danach p->handler()“ ist nicht als Textmuster erkennbar.
- Echte Bugs hängen von Kontroll-/Datenflüssen ab, die nur semantische Analyse (Datenfluss, Zeiger-Lifecycle) begreift.

Lernziele

- Die 4 Werkzeuge (Regex, sparse, smatch, Coccinelle) unterscheiden
- Warum Text-Matching für Datenfluss-Sicherheit nicht reicht

- Eigene Coccinelle-Regel für den UAF schreiben (Cross-Case-Trick)